

3/1错题

这一知识点主要考察 Linux 命令行中的 **演算子 (演算子/Operator)**，也就是如何根据前一个命令的执行结果（成功或失败）来控制后续命令的运行。

💡 知识点总结

使用 `||` (**OR 演算子**) 可以实现“只有当前一个命令执行**失败**（終了ステータスが 0 以外）时，才执行后一个命令”的逻辑。

🐣 小学生级说明：备用计划（Plan B）

想象你是一个正在执行任务的小机器人：

1. **Plan A (command1)**：我让你去买苹果。
2. “或者”指令 (`||`)：这个符号就像在说：“或者，万一没买到的话...”
3. **Plan B (command2)**：那你就去买香蕉。

所以，`command1 || command2` 的意思就是：如果买到了苹果（成功），任务结束；如果没买到苹果（失败），赶紧去买香蕉！

对比另一个常用符号：

- 如果你想说“买到苹果**之后**再去买香蕉（Plan A 成功才做 Plan B）”，那就要用 `&&`。

🔍 选项解析

- `||`：**正确答案**。只有在 `command1` 返回非零值（表示失败）时，才会执行 `command2`。
- `&&`：错误。这是在 `command1` **成功**时才执行 `command2`。
- `;`：错误。无论 `command1` 成功还是失败，都会**按顺序执行** `command2`。
- `$ / ?`：错误。这些通常用于变量引用（如 `$?` 查看上一个命令的状态码），不是连接命令的演算子。

✅ 正确答案

`||`

考试记忆口诀

- **&& (And)**: 成功**且**继续（双赢）。
- **|| (Or)**: 失败**或**补救（备胎）。
- **;** (**Semicolon**): 管他呢，**通通**执行。

这一知识点主要考察 Linux 中的 **正規表現（正規表現/Regular Expression）**。正規表現是一种用来在文本中进行模糊匹配的强大工具，类似于搜索时的“通配符”，但功能更精确。

知识点总结

在正規表現中，符号 **\$** 用来表示 **行末（ぎょうまつ）**。因此，要匹配以字符“a”结尾的行，正确的表达式是 **a\$**。

小学生级说明：给字符找“位置锚点”

想象每一行文字都是一条长长的跑道：

1. **开始的鸣枪声 (^)**: 如果你想找“第一个人是 A”的跑道，你会用 **^a**。符号 **^** 像个小尖头，指着行首。
 2. **结束的终点线 (\$)**: 如果你想找“最后一个人是 A”的跑道，你会用 **a\$**。符号 **\$** 就像终点线，挡在字符的后面。
 3. **其他的符号**:
 - **+** 和 **?** 是用来形容“数量”的（比如：有很多个 A，或者可能有一个 A）。
 - 它们不关心位置，只关心个数。
-

选项解析（结合图片表格）

- **a\$** : **正确答案**。根据表格，**\$** 表示“行末”。**a\$** 即匹配以 a 结尾的行。
 - **^a** : 错误。根据表格，**^** 表示“行头”。匹配以 a 开头的行。
 - **a+** : 错误。根据表格，这属于**拡張正規表現**，表示“a 出现 1 次以上”。
 - **a?** : 错误。根据表格，这属于**拡張正規表現**，表示“a 出现 0 次或 1 次”。
-

 **记忆大作战：如何记住图片里的表格？**

这张表内容很多，但我们可以通过**“分类闯关”**的方法快速记忆：

第一关：位置双子星（定位）

- ^ (Caret)**：想象是“向上”的箭头，指着一行的**开头**。
- \$ (Dollar)**：想象是钱袋子，我们要把钱装进袋子**最后**封口。

第二关：数量三兄弟（多少）

- * (Asterisk)**：0 次或多次（非常贪心，有没有都要）。
- + (Plus)**：1 次或多次（至少得有一个，Plus 代表增加）。
- ? (Question)**：0 次或 1 次（不确定有没有，最多只要一个）。

第三关：集合与转义（特殊处理）

- [] (Brackets)**：像个小篮子，里面放着好几个苹果，**挑其中一个**就行。
- .** (Dot)：像个橡皮泥，可以变成**任何一个**字符。
- \ (Backslash)**：变魔术的遮羞布。把有特殊能力的符号变回普通的字符（例如 `\$` 就只是一个普通的美元符号了）。

記号	説明	使用例（一致する文字）
.	任意の1文字	a.. aから始まる3文字 (aaa,abcなど)
*	直前の文字の0回以上の繰り返し	* 任意の文字列
[]	[] 内のいずれか1文字 []内では、以下の表現と併用可能 - 範囲指定 ^ 先頭にある場合は後続の文字以外	[abc] a,b,c のどれか1文字 [a-z] 小文字のアルファベット1文字 [^abc] a,b,c 以外のどれか1文字
^	行頭	^a aから始まる行
\$	行末	a\$ aで終わる行
\	次の1文字をエスケープ (通常の文字として処理)	* *という文字
+	直前の文字の1回以上の繰り返し 【拡張正規表現】	a+ aを1回以上繰り返す文字列 (a,aa,aaaなど)
?	直前の文字の0回もしくは1回の繰り返し 【拡張正規表現】	windows? windowsのsが0または1文字 (window,windows)
	左右いずれかの文字列 【拡張正規表現】	abc xyz abcまたはxyz

这一知识点主要考察 `sed` 命令的流编辑操作，特别是 **置換 (s命令)** 与 **文字变换 (y命令)** 的区别，以及如何同时进行**複数（多个）**编辑操作。

💡 知识点总结

实现多个字符的替换，可以使用 `sed` 的 **y 命令** 进行一对一的文字转换，或者使用多个 `-e` 选项连接多个 `s` 命令进行全局替换。

🐣 小学生级说明：给字母“变魔术”的两种方法

想象你手里有一本《test.txt》连环画，你想把里面的小写字母全部变大写：

1. “点对点” 变色镜 (y 命令):

- `y/ab/AB/` 就像是一个神奇的滤镜。它会盯着每一个字母看：如果看到 a 就把它变成 A，看到 b 就把它变成 B。
- 这是一种**一一对应**的关系：第一个位置的 a 对应第一个位置的 A，第二个位置的 b 对应第二个位置的 B。

2. “分步执行” 任务单 (-e 选项 + s 命令):

- `sed -e s/a/A/g -e s/b/B/g` 就像是给工人下了两个指令：
 - 第一个任务 (`-e`)：把所有的 a 变成 A。
 - 第二个任务 (`-e`)：把所有的 b 变成 B。
- `g` (Global) 的意思是不放过任何一个，全篇都要改。

为什么其他选项是错的？

- `s/ab/AB/g`：这个工人很死板，他只会去找连在一起的 "ab" 组合并把它们变成 "AB"。如果 a 和 b 没在一起（比如 "acb"），他就假装没看见，不会去改。
- `y/aA/bB/`：这会把 a 变成 b，把 A 变成 B，完全搞反了。

选项解析

- `sed y/ab/AB/ test.txt`：正确答案。`y` 命令用于字符转换 (Transform)，会将 ab 中的每个字符分别对应转换为 AB。
- `sed -e s/a/A/g -e s/b/B/g test.txt`：正确答案。使用 `-e` 选项可以指定多个编辑指令，`s/.../.../g` 实现全局置换。
- `sed s/ab/AB/g test.txt`：错误。这只会寻找字符串 "ab" 并替换为 "AB"，不会单独处理 a 或 b。
- `sed y/aA/bB/ test.txt`：错误。这会将 a 转换为 b，A 转换为 B，不符合题目要求。
- `sed /aA/bB/d test.txt`：错误。语法不正确，且 `d` 是删除命令。

✅ 正确答案 (全て選択)

- `sed y/ab/AB/ test.txt`
- `sed -e s/a/A/g -e s/b/B/g test.txt`

考试小秘籍

- 单个字母的一一对应 → 优先想 `y`。

- 复杂的字符串或多个独立逻辑 → 使用 `-e s/.../.../g`。
- 注意 `y` 命令不需要在末尾加 `g`，因为它天生就是对全行起作用的。

オプション		説明
<code>-e</code>		編集コマンドを指定 (編集コマンドが1つの場合は省略可)
<code>-f</code>		編集コマンドを記述したファイルを指定
編集コマンド		説明
sコマンド(置換)		
<code>s/文字列1/文字列2/</code>		各行の最初に現れる文字列1を文字列2に置換
オプション可能	<code>s/文字列1/文字列2/g</code>	<code>g</code> : 全ての文字列1を置換対象にする
	<code>s/文字列1/文字列2/i</code>	<code>i</code> : 文字列1の大文字と小文字を区別しない
	<code>s/文字列1/&を含む文字列2/</code>	<code>&</code> : 文字列1を参照し、置換結果に再利用する
dコマンド(行の削除)		
<code>/文字列/d</code>		文字列が含まれる行を削除
行番号1,行番号2 <code>d</code>		行番号1から行番号2までの行を削除
yコマンド(文字の一括置換)		
<code>y/文字1文字2.../ 文字3文字4.../</code>		文字1を文字3に、 文字2を文字4に置換

💡 知识点总结

实现多个字符的一对一替换，最简洁的方法是使用 `y` 命令；而要进行复杂的全局替换，则可以使用多个 `-e` 选项 连接 `s` 命令。

1.1 正規表現 (匹配工具箱)

这是用来描述你想找什么的语言：

- 定位符： `^` (行头) 和 `$` (行末)。记住：“头顶尖尖 (`^`)，末尾有钱 (`$`)”。
- 通配符： `.` 代表任意 1个 字符； `*` 代表前面那个东西重复 0次或多次。
- 集合： `[]` 是任选其一。如果 `^` 在 `[]` 里面，意思就变成了“排除”（例如 `[^abc]`）。
- 扩展版 (需注意)： `+` (1次以上) 和 `?` (0或1次) 是扩展正则，`sed` 使用时通常需要加 `-r` 参数。

1.2 sed 命令 (编辑手术刀)

这是找到内容后怎么处理的指令：

- `s` (Substitute)： 就像“搜索替换”。最常用，配合 `/g` 可以全行替换。
- `d` (Delete)： 直接把符合条件的行扔进垃圾桶。
- `y` (Transform)： 像“加密翻译机”。它不看单词，只看字母的位置，一对一翻译。

🧠 记忆大法：三步背诵法

想要记住这些复杂的符号，试试这个口诀和方法：

第一步：记“动作码” (Sed)

- **S** 是 **S**earch (搜索替换)。
- **D** 是 **D**elete (删除)。
- **Y** 是 **Y**is (一对一翻译 - 谐音记忆)。
- **-e** 是 **E**xtra (额外的指令)。

第二步：记“位置符” (RegEx)

- **^** (上箭头)：在行首。
- **\$** (美元符)：工资在月底发，所以是行末。

第三步：记“数量符” (RegEx)

- *****：像雪花，可能没有，也可能有无数片 (0~∞)。
- **+**：Plus 代表加法，至少得有 1 个 (1~∞)。
- **?**：问号代表不确定，有或没有，最多 1 个 (0~1)。

💡 知识点总结

在 `sed` 中，**并用 (併用可能)** 是指在执行 `s` (置换) 命令时，可以同时附加不同的选项 (如 `g`、`i`) 来改变其行为。而 `d` 命令用于整行删除，`y` 命令则用于字符的一一对应转换。

2. 什么是 `s` 命令的“并用”？

“并用”的意思就是：你可以在 `s/字符串1/字符串2/` 的最后面，像加调料一样同时加上多个后缀 (选项)，让它变得更强大。

- **g (Global)**：全行替换。不加 `g` 只换每行第一个。
- **i (Ignore case)**：忽略大小写。不管是 `a` 还是 `A` 都算匹配。
- **& (Reference)**：回溯引用。代表“刚才被选中的那个原始字符串”。

具体使用示例：

如果你想把文件里所有的 `apple` (不分大小写) 都换成 `apple(FRUIT)`，你可以这样并用 `g`、`i` 和 `&`：

```
sed 's/apple/&(FRUIT)/gi' test.txt
```

这里 `gi` 就是并用：既要全局替换 (`g`)，又要忽略大小写 (`i`)。而 `&` 代表把搜到的原词再填回去。

2. `d` 命令：行的削除 (删除手术刀)

d 命令不是替换里面的字，而是**直接把符合条件的“行”整行删掉**。

- 按字符串删：`/文字列/d`
 - `sed '/error/d' log.txt`（把所有带 "error" 字符的行都删了）。
- 按行号删：`行番号1,行番号2d`
 - `sed '1,3d' test.txt`（把第 1 到第 3 行全部删掉）。

3. **y** 命令：文字の一括置換（翻译转换机）

y 命令最容易和 **s** 搞混。它是一对一的翻译，不看单词，只看位置。

- 用法：`y/源字符列表/目标字符列表/`
- 特点：源列表的第 1 个字符换成目标列表的第 1 个，第 2 个换成第 2 个，以此类推。

具体例子：

```
sed 'y/abc/123/' test.txt
```

- 所有的 **a** 变成 **1**
- 所有的 **b** 变成 **2**
- 所有的 **c** 变成 **3**

注意：**y** 不能像 **s** 那样加 **g** 后缀，因为它默认就是对全行所有字符生效的。

记忆表格的秘密武器

为了帮你背下 [image_cdb703.png] 这张表，我们给这些命令编个身份：

命令字符	身份	记忆口诀（中文 + 日语）
s	搜索替换师	Substitute。最常用的 置換 ，记得加 g 才是全部换。
d	行销毁员	Delete。整行 削除 ，看谁不顺眼就删谁。
y	密码翻译官	两个列表对对碰，一字换一字的一 括置換 。
-e	多面手	Extra。想一次干好几件事，就用多个 -e 。

这一知识点主要考察 **sed** 命令中如何指定**范围（範囲指定）以及执行删除（行の削除）**操作。

知识点总结

在 **sed** 中，使用 **d** 命令来删除行，并用**逗号（,）** 连接起始行号和结束行号来表示一个闭区间范围。因此，删除第 1 到第 3 行的正确语法是 **1,3d**。

🐼 小学生级说明：给“行”画一个圈

想象 `sed` 是一个拿着橡皮擦的机器人，你正在给它下指令：

- 1. 确定动作 (d):
- 2. 你想让它“擦掉”（**删除**），所以动作代码是 `d` (Delete)。
- 3. 确定范围 (1,3):
 - 如果你说 `1d`，它只擦第 1 行。
 - 如果你想让它擦掉从第 1 行到第 3 行，你需要用**逗号** `,` 把它们连起来。这个逗号就像是在说“从这里到那里”。
 - **注意：**千万不要用分号 `;` 或者斜杠 `/`，它们在 `sed` 的世界里有不同的用途（比如分号是分开两个不同的任务，斜杠是用来寻找特定的单词）。

🔍 选项解析

- `sed 1,3d test.txt`：**正确答案**。根据表格，`行番号1,行番号2d` 是删除从行号 1 到行号 2 的标准写法。
- `sed 1;3d test.txt`：错误。这会被解释为两个独立的命令：第一个是 `1`（不做任何事），第二个是 `3d`（只删除第 3 行）。
- `sed 1/3d test.txt`：错误。斜杠通常用于包含字符串的匹配模式，不能这样连接行号。
- `sed s/1/3/g test.txt`：错误。这是要把数字 "1" **替换** 成 "3"，而不是删除行。

✅ 正确答案

`sed 1,3d test.txt`

📝 记忆强化

结合你提供的图片，我们可以把 `d` 命令的用法总结为两类：

用法	描述	例子
按内容删	/文字列/d	<code>sed '/error/d'</code> (删掉带 error 的行)
按行号删	开始行,结束行d	<code>sed '1,3d'</code> (删掉第 1 到 3 行)

这一知识点主要考察 `split` 命令的语法格式，即如何指定**分割行数**、**输入文件**以及**输出文件的前缀**（プレフィックス）。

💡 知识点总结

使用 `split` 命令分割文件时，标准语法格式为 `split [选项] [输入文件] [输出文件前缀]`。
要按 500 行分割并指定文件名前缀为 `hoge`，应使用 `split -500 file hoge`。

👶 小学生级说明：把大蛋糕切成小块

想象你有一个巨大的**蛋糕 (file)**，你想把它切成每块 500 层的厚度，并且在每块切好的蛋糕盒上贴上 **hoge** 的标签：

1. **确定切法 (-500)**：你首先告诉机器人：“每 500 行切一刀”。
2. **选择目标 (file)**：接着指着那个大蛋糕说：“切这个文件”。
3. **贴上标签 (hoge)**：最后告诉它：“切出来的碎块，名字开头都给我叫 `hoge`”。
 - 机器人很聪明，它会自动在后面加上 `aa`，`ab`，`ac` ... 来区分，变成 `hogeaa`，`hogeab`

注意顺序：在 Linux 命令里，通常是“**怎么做 → 谁被做 → 产出叫什么**”。所以 `hoge` 必须放在最后。

🔍 选项解析

- `split -500 file hoge`：**正确答案**。符合 `[选项] [输入文件] [前缀]` 的标准格式。
- `split -500 hoge file`：错误。这会让机器人以为你要分割一个叫 `hoge` 的文件，并把产出命名为 `fileaa`。
- `split file hoge -500`：错误。选项 `-500` 必须放在前面，不能甩在最后。
- `split -500 file hogeaa hogeab`：错误。你只需要提供**前缀** `hoge`，后面的 `aa`，`ab` 是系统自动生成的，不能手动一个个写出来。

✅ 正确答案

`split -500 file hoge`

📝 考试小秘籍

- **行数指定**：可以写成 `-l 500` 或者简写成 `-500`。
- **后缀规律**：默认是字母后缀 (`aa`，`ab` ...)。如果你想用数字后缀 (`00`，`01` ...)，需要加上 `-d` 选项。

这一知识点主要考察在 Linux 中创建 **ext3 文件系统（ファイルシステムの作成）** 的多种等价命令。

💡 知识点总结

创建 **ext3** 文件系统可以使用通用的 `mkfs` 命令通过 `-t` 指定类型，也可以使用专门的 `mke2fs` 命令并配合 `-j`（代表 Journaling，即 ext3 的日志功能）或 `-t ext3` 选项来实现。

🐣 小学生级说明：给硬盘贴上“格子标签”

想象你刚买了一块白板（硬盘分区 `/dev/sda2`），你想在上面画出特定的格子（文件系统）来存放文件。

1. **通用工具法 (`mkfs`)**：就像拿出一个全能工具箱，说：“我要做一个 **ext3** 类型的格子。”
 - 命令：`mkfs -t ext3 /dev/sda2`
2. **专门工具法 (`mke2fs`)**：`mke2fs` 本身是为 ext2 设计的。但 ext3 其实就是“带日志功能的 ext2”，所以：
 - **方法 A**：告诉它“加个日志 (`-j`)”，它就变成了 ext3。
 - **方法 B**：直接明确告诉它“类型是 ext3 (`-t ext3`)”。

为什么其他选项是错的？

- `mkfs ext3 ...`：中间漏掉了关键的选项开关 `-t`。
- `mke2fs /dev/sda2`：如果不加任何选项，它默认会创建一个 **ext2**（没有日志功能）的文件系统。

🔍 选项解析

- `mke2fs -j /dev/sda2`：**正确答案**。`-j` 选项代表 **Journaling**，会将 ext2 升级为具有日志功能的 ext3。
- `mkfs -t ext3 /dev/sda2`：**正确答案**。这是最标准、最通用的格式。
- `mke2fs -t ext3 /dev/sda2`：**正确答案**。在现代系统中，`mke2fs` 也支持直接用 `-t` 指定类型。
- `mkfs ext3 /dev/sda2`：**错误**。语法不完整，必须写成 `mkfs.ext3`（带点）或者 `mkfs -t ext3`。
- `mke2fs /dev/sda2`：**错误**。默认创建的是 ext2。

✔ 正确答案（全て選択）

- mke2fs -j /dev/sda2
- mkfs -t ext3 /dev/sda2
- mke2fs -t ext3 /dev/sda2

📝 考试小秘籍

- ext2 → 直接用 `mke2fs`。
- ext3 → `mke2fs -j` 或 `mkfs -t ext3`。
- ext4 → `mkfs -t ext4` 或 `mke2fs -t ext4`。

オプション	説明
-j	ext3ファイルシステムを作成
-t ファイルシステムの種類	ファイルシステムの種類を指定 (ext2/ext3/ext4)
-c	ファイルシステムを作成する前に不良ブロックを検査

mke2fs [オプション] デバイス名

オプション	説明
-t ファイルシステムの種類	ファイルシステムの種類を指定
-c	ファイルシステムを作成する前に不良ブロックを検査

mkfs [オプション] デバイス名

这一部分主要考察 Linux 文件系统创建工具 `mke2fs` 和 `mkfs` 的基本书式（格式）与主要选项。

💡 知识点总结

创建文件系统的核心在于：使用 `mkfs -t` 或 `mke2fs -t` 来指定具体的文件系统类型（如 `ext2/ext3/ext4`），其中 `mke2fs -j` 是创建 `ext3` 的特有快捷方式。

📊 图片表格深度讲解

我们要把这几张表拆解为两个部分：“创建命令的逻辑”和“常用的功能开关”。

2.1 mke2fs 专用选项

`mke2fs` 本身是为 ext 家族量身定制的工具。

- `-j` (**Journaling**): 这是最常考的。它会给分区加上“日志”，直接把 ext2 升级成 **ext3**。
- `-t [类型]`: 直接指定你想创建的是哪一代，比如 `ext4`。
- `-c` (**Check**): 在动工盖房子之前，先检查一下地基有没有**不良ブロック**（坏块）。

2.2 mkfs 通用语法

`mkfs` 像是一个通用的调度员，它的参数非常直观。

- `-t [类型]`: 通过这个参数，调度员会去叫醒具体的子工具（如 `mkfs.ext4` 或 `mkfs.xfs`）来干活。
- `-c`: 功能和上面一样，也是检查坏块。

记忆大作战：如何背下这些参数？

为了让你过目不忘，我们给这些字母编个故事：

第一关：记“J”和“T”（核心类型）

- `-j` → **Journal** (日志): 想象 ext3 是一本带锁的**日记本 (Journal)**，所以创建 ext3 必须加 `-j`。
- `-t` → **Type** (类型): 不管是 `mkfs` 还是 `mke2fs`，只要你想指定 **Type**，就用 `-t`。

第二关：记“C”（体检）

- `-c` → **Check** (检查): 在创建系统前，先给硬盘做个“体检”，看看有没有 **Corrupt**（损坏）的**不良ブロック**。

第三关：记“e”（额外任务）

- 回到 `sed` 的那张图：`-e` 代表 **Extra**（额外的编辑指令）。当你有很多置换任务时，每个任务前面都要加个 `-e`。

快速对照背诵表

命令	关键选项	功能 (日本語)	对应文件系统
mke2fs	-j	ext3ファイルシステムを作成	ext3
mke2fs	-t [type]	種類を指定 (ext2/3/4)	指定的类型
mkfs	-t [type]	種類を指定	指定的类型
两者皆有	-c	不良ブロックを検査	N/A

🤖 避坑指南

- 注意！ 图片中显示 `mkfs` 也是支持 `-c` 的，但在某些特定的文件系统（如 XFS）下，`mkfs` 的参数格式会有变化。考试时请锁定 `-t` 作为最稳妥的类型指定方式。

コマンド	説明
yy Y	カーソル位置の行をバッファにコピー
yw	カーソル位置の単語をバッファにコピー
dd	カーソル位置の行を削除し、バッファにコピー
dw	カーソル位置の単語を削除し、バッファにコピー
x	カーソル位置の文字を削除し、バッファにコピー
X	カーソル位置の左の文字を削除し、バッファにコピー
p	バッファの内容を挿入(文字や単語はカーソルの右、行はカーソルの下に挿入)
P	バッファの内容を挿入(文字や単語はカーソルの左、行はカーソルの上に挿入)
u	元に戻る(直前の編集をとりやめる)
.	繰り返す(直前の編集を再実行する)

这一知识点主要考察 **viエディタ (vi编辑器)** 在 **コマンドモード (命令模式)** 下用于提高编辑效率的快捷指令，特别是如何快速重复上一次的操作。

💡 知识点总结

在 vi 的命令模式下，按下 **点号 (.)** 即可****繰り返す (重复) ****执行上一次进行的编辑操作（如删除、粘贴等）。

🔍 题目与图片解析

根据你上传的图片 `image_cda49b.png`，我们可以清楚地看到最后一个指令：

- `.` (ドット): 繰り返す (直前の編集を再実行する)
 - 含义：如果你刚才用 `dd` 删了一行，现在只要按一下 `.`，它就会再删掉当前的一行，不需要再敲一次 `dd`。

其他选项解析：

- `u` (Undo): 根据图片，它是 元に戻す (撤销)，即取消刚才的编辑。
 - `x` : 根据图片，它是 文字を削除 (删除光标处的字符)。
 - `j` 和 `k` : 这两个不在图中，但它们是 vi 中最基础的移动键 (`j` 向下，`k` 向上)，不属于编辑指令。
-

图片表格深度讲解：vi 常用命令分类

为了帮你背下 `image_cda49b.png` 这张表，我们把它按功能逻辑拆解：

2.3 复制与剪切 (Copy & Delete to Buffer)

- `yy` 或 `Y` : 拷贝整行。
- `yw` : 拷贝一个单词 (word)。
- `dd` : 删除整行 (实际上是剪切到缓冲区)。
- `dw` : 删除一个单词。
- `x` 和 `X` : 删除单个字符。小写 `x` 删右边 (光标处)，大写 `X` 删左边。

2.2 粘贴 (Paste)

- `p` (小写): 往下或往右贴 (像是在后面追加)。
- `P` (大写): 往上或往左贴 (抢占前面的位置)。

2.3 效率与后悔药

- `u` (Undo): 后悔了，点一下回到过去。
 - `.` (Repeat): 觉得刚才的操作很爽，再来一次！
-

记忆大法：动作联想法

vi 的命令非常讲究“对称”和“首字母”，试着这样记：

- `u` 是 "Undo": 撤销，每个人都知道。
- `.` 是 "Ditto" (同上): 在英语里点号常表示“同上”，所以它代表重复。

- **p** 是 "Paste": 粘贴。
- **y** 是 "Yank": 在英语里有“用力拔/拉”的意思，想象把文字“拉”到剪贴板里（拷贝）。
- **大小写的区别**: 在 vi 里，**大写通常比小写更“强势”**。小写 **p** 贴在后面，大写 **P** 就要贴在前面；小写 **x** 删脚下的，大写 **X** 还要删左边的。

✓ 正确答案

.

 练习建议

你可以尝试在 vi 中先输入 `dd` 删掉一行，然后疯狂点击 `....`，你会看到行像多米诺骨牌一样连续消失，这样你就能深刻记住这个“点”的威力了。

オプション	説明
async	非同期で入出力を行う
auto	「mount -a」コマンド実行時にマウント
noauto	「mount -a」コマンド実行時にマウントしない
defaults	デフォルト指定 (async, auto, dev, exec, nouser, rw, suid)
exec	バイナリの実行を許可
noexec	バイナリの実行を禁止
ro	読み取り専用
rw	読み書きを許可
suid	SUIDとSGIDを有効化
user	一般ユーザでマウント可、本人のみアンマウント可
users	一般ユーザでマウント可、誰でもアンマウント可
nouser	一般ユーザによるマウントを禁止

`/etc/fstab` 挂载选项全解析

这一张表是 LPI 考试的常客，主要涉及“权限”和“自动化”。

オプション (选项)	说明 (中文)	核心记忆点 (日本語/English)
async	非同期输入输出	Async = 非同期。不立刻写硬盘，先存内存。
auto	自动挂载	mount -a 时自動マウント。
noauto	不自动挂载	No + Auto。手动才干活。
exec	许可执行程序	Execute = 実行。允许跑程序。
noexec	禁止执行程序	No + Exec。为了安全，不准跑程序。
ro	只读	Read Only = 読み取り専用。
rw	读写	Read Write = 読み書き許可。
suid	有效化 SUID	让普通人临时有老板权限。
user	用户可挂载	本人のみ卸载。单数 user = 1个人。
users	用户可挂载	誰でも卸载。复数 users = 大家。
nouser	禁止用户挂载	No + User。只有 Root 能动。
defaults	默认礼包	包含：async, auto, dev, exec, nouser, rw, suid。

这一知识点主要考察 `/etc/fstab` 文件中 **マウントオプション (挂载选项)** 的具体含义，特别是关于**用户权限控制**的设置。

💡 知识点总结

在 `/etc/fstab` 中，使用 `nouser` 选项可以明确禁止（禁止）一般用户进行挂载操作，这是系统的默认安全行为。

🔍 题目与图片解析

根据你提供的图片 `image_cda3ff.png`，我们可以精准锁定关于用户权限的三个核心选项：

- `user`：允许一般用户挂载，但只有挂载的那个人才能卸载。
- `users`：允许一般用户挂载，且任何人都可以卸载。
- `nouser`：一般ユーザによるマウントを禁止（禁止一般用户挂载）。

其他选项解析：

- `noexec`：禁止执行二进制文件（バイナリの実行を禁止）。
- `noauto`：在执行 `mount -a` 时不自动挂载。

📊 图片表格深度讲解：`/etc/fstab` 挂载选项分类

为了帮你背下 `image_cda3ff.png`，我们将这些选项按“功能组”拆解：

2.4 权限组（谁能动？）

- `user` / `users`：给一般用户开绿灯。
- `nouser`：只准 Root 动，一般用户靠边站。

2.2 行为组（怎么动？）

- `auto` / `noauto`：开机或 `mount -a` 时是否自动干活。
- `ro` (Read Only)：只能看，不能改。
- `rw` (Read Write)：能看也能改。

2.3 执行组（能运行吗？）

- `exec` / `noexec`：这个分区里的程序准不准运行。

2.4 大礼包选项

- `defaults`：这是一个“全家桶”，包含了 `async`，`auto`，`dev`，`exec`，`nouser`，`rw`，`suid` 这七个常用设置。

🧠 记忆大法：前缀联想法

在 Linux 的选项里，`no-` 前缀通常代表“不”或“禁止”。

- `nouser` → No + User → 不给一般 User 用。
- `noexec` → No + Execute → 不准执行。
- `noauto` → No + Auto → 不自动。

背诵口诀：

User 允许，Users 大家许，No-user 绝对不准许。Ro 只能读（Read Only），Rw 读写都可以。

✅ 正确答案

`nouser`

📝 考试小秘籍

考试中经常会问 `defaults` 包含了哪些选项。请一定要记住：`defaults` 默认是禁止一般用户挂载的，也就是说它包含了 `nouser` 而不是 `user`。

这一知识点主要考察 `シェルスクリプトの実行`（Shell Script 的执行方式），特别是当文件没有 `実行権限`（执行权限）时的处理方法。

💡 知识点总结

当脚本没有执行权限（`chmod +x` 未设置）时，不能直接通过路径调用，必须通过 **シェル（Shell）命令** 直接加载运行，或者使用 `source` / `.` 命令在当前进程中执行。

🔍 选项解析与图片关联记忆

根据你提供的图片信息，我们来拆解为什么这三个是正确答案：

- 1. `. sample.sh`：正确答案。
 - 说明：这里的 `.` 是 `source` 命令的缩写。它不需要文件有执行权限，会直接读取脚本内容并在当前 Shell 中执行。
- 2. `source sample.sh`：正确答案。
 - 说明：与 `.` 完全等价。常用于让修改后的配置文件（如 `.bashrc`）立即生效。
- 3. `bash sample.sh`：正确答案。
 - 说明：这是直接调用 **bash 解释器** 来读取并运行脚本。因为是 bash 进程在读取文件，所以脚本文件本身只需要有“读取权限（r）”，不需要“执行权限（x）”。

错误选项解析：

- `./sample.sh`：错误。只有设置了执行权限位（Executable bit）时，这种通过路径指定的方法才有效。由于题目明确说“未设定执行权”，执行此命令会报错 `Permission denied`。
- `run sample.sh`：错误。Linux 中没有内置名为 `run` 的命令来执行脚本。
- `export sample.sh`：错误。`export` 是用来设置环境变量的，不能用来运行脚本。

📊 关联表格背诵：执行方式大对比

为了帮你记牢，我们结合之前关于 `sed` 和 `/etc/fstab` 的记忆逻辑，给执行方式分个类：

执行方式	权限要求	进程行为	常用场景
<code>bash script</code>	仅需 读取 (r)	开启子进程运行	普通脚本运行
<code>source</code> 或 <code>.</code>	仅需 读取 (r)	在当前进程运行	环境设定生效
<code>./script</code>	必须有 执行 (x)	开启子进程运行	已经封装好的工具

🧠 记忆大作战：如何区分 `source` 和 `bash`？

- `bash` 是“外包”：请一个叫 `bash` 的工人来读脚本，工人有权限读就行，脚本不需要自己会动。
- `source` 是“吸收”：就像把脚本里的内容直接吸进当前的 Shell 里，让它变成自己的一部分。
- `.` (点) 是“简写”：想象这个点是一个注射器，把脚本内容直接注射到现在的 Shell 环境中。

✅ 正确答案（3つ選択）

- `. sample.sh`
- `source sample.sh`
- `bash sample.sh`

📝 考试小秘籍

题目中只要提到“**実行権がない**”（没有执行权），就绝对不能选以 `./` 或绝对路径开头的选项。必须找前面带着 `bash`、`sh`、`source` 或 `.` 的选项。

这一知识点主要考察 `tr` コマンド 的功能选项，特别是如何处理**連続した文字（连续字符）**以及 Linux 的 **リダイレクト（重定向）** 操作。

`tr [オプション] [文字列1 [文字列2]]`

オプション	説明
<code>-d</code>	文字列1で指定した文字を削除
<code>-s</code>	文字列1で指定した文字が連続した場合、1文字に置き換える

💡 知识点总结

`tr -s` 命令用于将输入文本中**连续重复的指定字符压缩（Squeeze）为一个**。配合重定向符号 `<` 和 `>`，可以实现从源文件读取并在处理后输出到目标文件。

🐼 小学生级说明：把“一长串”挤成“一个”

想象你有一台**挤压机 (tr -s)**：

1. 原材料 (`< file1`)：
2. 你从一个叫 `file1` 的仓库里拿出一根长绳子，上面串了很多珠子。
3. 加工规则 (`'#'`)：
4. 你告诉挤压机：“如果看到连续在一起的好几个 `#` 号，就给我使劲儿挤！”

- 5. 加工结果 (`-s`):
- 6. 原本是 `###` 的地方，被挤成了只有一个 `#`。
- 7. 存入新仓库 (`> file2`):
- 8. 挤好的绳子最后存进了名为 `file2` 的新仓库里。

注意方向：

- `<` 是把文件内容推给命令。
- `>` 是把命令的结果存入文件。

🔍 选项解析

- 「file1」ファイル内の連続した「#」を1つに置き換えて「file2」ファイルに出力する：正确答案。
 - `-s` (Squeeze)：压缩连续字符。
 - `< file1`：输入源。
 - `> file2`：输出目标。
- 「file1」内の「#」を削除する：错误。删除字符需要使用 `-d` 选项。
- 「file2」内の...「file1」に出力する：错误。重定向符号的方向弄反了。

📊 关联表格背诵： `tr` 命令的常用“变身”

为了帮你记全 `tr` 的用法，请记住这三个核心字母：

选项 (Option)	功能 (日本語)	记忆口诀 (中文)
<code>-s</code>	連続する文字を1つにまとめる	Squeeze (挤压) 成一个。
<code>-d</code>	文字を削除する	Delete (删除) 消失。
<code>-c</code>	セット1にない文字を対象にする	Complement (反选)。

✅ 正确答案

「file1」ファイル内の連続した「#」を1つに置き換えて「file2」ファイルに出力する

 考试小秘籍

- 看到 `-s` → 找 “連続” 和 “1つ”。
- 看到 `-d` → 找 “削除”。
- `<` 和 `>` 的顺序：永远是 `命令 < 输入 > 输出`。

这一知识点主要考察 `od` (Octal Dump) 命令 的显示格式选项。虽然 `od` 默认以八进制显示，但通过特定选项可以将其转换为 **ASCII文字** 或 **エスケープ文字**（转义字符，如 `\n`，`\t`）。

`-t` オプション

フォーマット	説明
c	ASCII文字またはバックスラッシュ付きのエスケープ文字で表示。
o[SIZE]	8進数で表示。SIZEで表示を区切るバイト数を指定できる。 (未指定時4バイト)
x[SIZE]	16進数で表示。SIZEで表示を区切るバイト数を指定できる。 (未指定時4バイト)

这一知识点主要考察 `od` (Octal Dump) 命令 如何通过 `-t` オプション 来灵活指定数据的 **出力形式**（输出格式）。

 知识点总结

`od -t` 后面紧跟的字母决定了文件内容以何种“语言”显示。根据图片 `image_cd94b7.png`，最核心的三个格式是 `c` (字符)、`o` (八进制) 和 `x` (十六进制)。

 图片内容深度解析 (image_cd94b7.png)

这张表解释了 `-t` 选项后可以接的 **フォーマット**（格式）：

- `c` (Character):
 - 说明：以 **ASCII文字** 或带反斜杠的 **エスケープ文字**（如 `\n`，`\t`）显示。
 - 记忆点：你想“看懂”文件里的文字时用它。
- `o` (Octal):
 - 说明：以 **8進数** 显示。

- 进阶：后面可以跟 [SIZE]（如 o2，o4）来指定多少个字节切分一次。默认是 4 字节。

3. x (Hexadecimal):

- 说明：以 16 進数 显示。
- 进阶：同样可以指定 [SIZE] 来控制切分单位。

💡 知识点总结

要以 ASCII 文字 和 エスケープ文字 显示文件内容，最常用的选项是 -c，而在现代 od 版本中，功能等价且符合新标准的写法是 -t c。

🔍 选项解析

- c：正确答案。这是最传统的选项，用于以 ASCII 字符或转义序列显示字节。
- t c：正确答案。-t (Type) 选项用于指定输出格式。c 代表 Character，即以 ASCII/转义字符形式显示。
- x / -t x：错误。这会以 十六进制 (Hexadecimal) 显示内容。
- o / -t o：错误。这会以 八进制 (Octal) 显示内容（这是 od 的默认行为）。

od [オプション][ファイル名]

オプション	説明
-t	出力フォーマットを指定
-o (デフォルト)	8進数2バイト区切りで表示 (-t o2と同じ)
-x	16進数2バイト区切りで表示 (-t x2と同じ)
-c	ASCII文字またはバックスラッシュ 付きのエスケープ文字で表示 (-t cと同じ)

📊 od 命令常用输出格式对应表

为了帮你彻底记住这一类题目，可以将选项和对应的进制/格式建立关联：

选项 (短)	选项 (长/标准)	输出格式 (日本語)	记忆口诀
-c	-t c	ASCII文字 / エスケープ文字	Character (字符)
-x	-t x	16進数	Hex (十六进制)
-o	-t o	8進数 (默认)	Octal (八进制)
-d	-t u	10進数 (无符号)	Decimal (十进制)

🧠 记忆大法：首字母关联

- C 是 Character（字符）：看到 ASCII 文字就找 `-c`。
- X 是 Hex（16进制）：看到 16 进制就找 `-x`。
- O 是 Octal（8进制）：看到 8 进制就找 `-o`。

✅ 正确答案（2つ選択）

- -t c
- -c

📝 考试小秘籍

在 LPI 考试中，很多命令（如 `od`，`mkfs` 等）都有“传统简写选项”和“`-t` 开头的标准选项”。通常题目要求多选时，这两种形式往往都是正确答案。

这一知识点主要考察 **man ページのセクション (Section)** 的结构。当直接运行 `man crontab` 时，系统默认会显示第 1 节（ユーザーコマンド/用户命令）的内容，而要查看 `/etc/crontab` 这种設定ファイル（配置文件）的格式，则必须指定第 5 节。

💡 知识点总结

`man` 页面按内容分类存储在不同的节中。`crontab` 既是一个命令（Section 1），也是一个配置文件的格式（Section 5）。若不指定节编号，`man` 会优先显示编号最小的页面（通常是命令说明）。

🔍 题目解析与图片说明

1. 出现该现象的原因：manページのセクション番号を指定していない（正确答案）。

- 观察图片 `image_cd425f.png`，第一行显示的是 `CRONTAB(1)`。
- 括号里的 `(1)` 代表这是 **User Commands (用户命令)** 的说明，即 `crontab` 命令的使用方
- 法因为你没有告诉 `man` 你想看哪一部分，它默认带你去了第一部分。

```
$ man crontab
CRONTAB(1)                                     User Commands
CRONTAB(1)

NAME
    crontab - maintains crontab files for individual users

SYNOPSIS
    crontab [-u user] file
    crontab [-u user] [-l | -r | -e] [-i] [-s]
    crontab -n [ hostname ]
    crontab -c
(省略)
```

2. 获取正确信息的命令：`man 5 crontab`（正确答案）。

- 为了查看 `/etc/fstab` 或 `/etc/crontab` 等文件的**书写格式**，需要访问第 **5** 节（File Formats）。

 man 页面节编号速查表 (背诵重点)

为了帮你背下这个 Linux 考试必考点，请记住这几个核心数字：

节编号 (Section)	内容分类 (日本語)	记忆口诀 (中)	セクション番号	内容
1	ユーザーコマンド	1 类人 (普通人)	1	ユーザーコマンド
5	ファイルの書式	5 (格式) —— 配置文件	2	システムコール (カーネルが提供する関数)
8	システム管理コマンド	8 (吧) —— 只有 root 能用	3	ライブラリ呼び出し (プログラムライブラリに含まれる関数)
			4	特殊ファイル (通常 /dev 配下に存在するファイル)
			5	ファイルの書式と慣習 (例: /etc/passwd)
			6	ゲーム
			7	その他いろいろなもの (マクロパッケージや慣習などを含む) 例えば man(7) や groff(7)
			8	システム管理コマンド (通常は root 用)
			9	カーネルルーチン [非標準]

オプション	説明
-k	キーワードに一部一致するコマンドやファイルの man ページを一覧表示 (apropos コマンドと同じ)
-f	キーワードに完全一致するコマンドやファイルの man ページを一覧表示 (whatis コマンドと同じ)

`man [オプション] [セクション番号] キーワード`

💡 知识点总结

`man` 命令不仅能通过 `-k` 或 `-f` 进行关键字搜索，还可以通过 **セクション番号 (节编号)** 快速定位到不同类别的帮助信息（如命令、文件格式、管理工具等）。

2.5 検索オプション (搜索选项)

当你不确定命令的全名，或者想看所有相关的帮助时：

- `-k` (**apropos**): 关键词 **部分一致**。就像在百度搜模糊词，它会列出所有包含该词的页面。
- `-f` (**whatis**): 关键词 **完全一致**。只显示名字一模一样的页面摘要。

🧠 记忆大作战：如何分清 1 和 5？

- `man crontab` (默认 1): 你想知道“怎么执行这个命令”（例如 `-e` 编辑, `-l` 列表）。
- `man 5 crontab` (指定 5): 你想知道“文件里怎么写日期和星号”（例如 `* * * * *`
`root command`）。

✅ 正确答案 (2つ選択)

- `man`ページのセクション番号を指定していない
- `man 5 crontab`

📝 考试小秘籍

题目中只要提到想查“**ファイルの書式**” (文件格式) 或“**/etc/... 的内容**”，脑子里立刻要蹦出数字 5。如果是想查“**管理者用コマンド**”，则锁定数字 8。

这一知识点主要考察 **正規表現 (正则表达式)** 中 **点号 (.)** 的基本含义。

💡 知识点总结

在正则表达式中，**点号 (.)** 代表**任意 1 个字符 (任意的1文字)**。因此，`grep '1234.'` 的意思是：寻找包含“1234”且其后紧跟**至少一个字符**的行。

🐣 小学生级说明：找带着“小尾巴”的 1234

想象你在玩找人游戏，你要找的是“1234”这个人，但他必须**牵着一个小伙伴**（任意字符）。

1. 规则 (`1234.`):

- 1234 是固定名字。
- . 是一个**“隐身衣”，它可以是任何字符（数字、字母、符号都行），但必须占一个位置**。

2. 谁符合条件？

- 12344：有 1234，后面跟着个 4（小伙伴），OK!
- 123445：有 1234，后面跟着个 4（后面还有个 5 没关系，只要满足 1234+一个字符就行），OK!
- 0123499：中间有 1234，后面跟着个 9，OK!

3. 谁不符合条件？

- 123：连 1234 都凑不齐，NG!
- 如果有一行正好就是 1234（后面啥也没有），那也不行，因为那个点号 . 要求必须得有一个“小伙伴”。

✅ 正确答案（出力される行）

根据 test.txt 的内容，只要包含 1234 且后面还有字符的行都会被抓出来：

- 12344
- 123445
- 0123499
- 0123499
- 123445
- 12344

关联表格背诵：正则符号的小秘密

为了帮你背下这张正则表，我们给符号们起个绰号：

符号	绰号	作用 (日本語)
.	隐身衣	任意の1文字にマッチ
*	复读机	直前の文字の0回以上の繰り返し
^	尖帽子	行の先頭
\$	小尾巴	行の末尾
[]	选秀台	カッコ内のいずれかの1文字

考试小秘籍

如果题目是 `1234*`（用星号），那 `123` 还是不行，但如果原本就有 `1234` 就可以，因为星号代表前面的 `4` 可以出现 0 次或多次。但点号 `.` 非常霸道，**必须得占一个位置**。

这一知识点主要考察用于处理文本中**重複（重复）**内容的工具。

知识点总结

在 Linux 中，`uniq` 命令用于报告或省略文件中的**重複した行（重复行）**。需要特别注意的是，该命令只能处理**連続して重複した行（相邻的重复行）**，因此通常会先配合 `sort` 命令使用。

小学生级说明：神奇的“消消乐”按钮

想象你有一排彩色的珠子（文件里的行），其中有些珠子颜色是一模一样的。

1. **前提条件（ソート）：**
 2. 你的珠子必须先排好序（比如红色跟红色在一起，蓝色跟蓝色在一起）。如果珠子乱七八糟的，这个按钮就不灵了。
 3. **按下按钮（`uniq`）：**
 4. 当你按下 `uniq` 按钮，它会看一眼相邻的两颗珠子。如果它们一模一样，它就会把后面那颗“变透明”，最后让你只看到**一颗红色的和一颗蓝色的**。
 5. **其他按钮是干嘛的？**
 - `cat`：像个手电筒，只是把珠子照出来给你看。
 - `head`：像个小盖子，只让你看最前面的几颗。
 - `nl`：像个记号笔，给每颗珠子写上数字编号（Numbering Lines）。
 - `sort`：像个整理员，负责把珠子按顺序排好，但它不会把重复的变没。
-

选项解析

- `uniq`：**正确答案**。专门用来合并或删除连续重复行的工具。
- `nl`：用于在行首添加行号。
- `sort`：用于排序，虽然 `sort -u` 也可以实现去重，但题目问的是专门用于合并连续重复行的命令。
- `head`：用于显示文件开头的几行。
- `cat`：用于连接文件并显示内容。

✔ 正确答案

uniq

📊 关联表格背诵：文本处理的小工具箱

为了帮你记牢，我们结合之前背过的工具做一个对比：

命令	核心功能 (日本語)	记忆口诀
uniq	重複した行を1行にまとめる	Unique（独特），只留一个。
sort	行を並べ替える	Sort（分类/排序）。
tr -s	連続した文字を1つにまとめる	Squeeze（挤压）重复字母。
sed d	行を削除する	Delete（删除）。

📝 考试小秘籍

题目中只要出现 “連続して重複” 和 “1行にまとめる”，标准答案一定是 `uniq`。如果题目问如何统计重复了多少次，记得选 `uniq -c`（count）。

这一知识点主要考察 Linux 文件系统中 **ジャーナリング（Journaling/日志功能）** 的有无。

ファイルシステム	説明
ext2	以前のLinuxで標準的に使用されていた規格
ext3	・ext2の後継 ・ジャーナリングファイルシステム
ext4	・ext3の後継 ・ジャーナリングファイルシステム
XFS	・シリコングラフィクス社(SGI)が開発 ・ジャーナリングファイルシステム ・動的inode
JFS	・IBMが開発 ・ジャーナリングファイルシステム ・動的inode

💡 知识点总结

ext2 是一个非日志文件系统（ジャーナリングではない）。而 **ext3**、**ext4**、**XFS** 和 **JFS** 都具备日志功能，能够在系统意外宕机时更快地恢复数据一致性。

🐼 小学生级说明：带不带“记事本”的区别

想象文件系统是一个大仓库，管理员要把货物搬进搬出。

- 1. 没有日志的仓库 (ext2):
- 2. 管理员直接搬货。如果搬到一半突然断电了，管理员会忘记刚才搬到哪了。下次开机时，必须把整个仓库从头到尾检查一遍（执行 `fsck`），非常费时间。
- 3. 带日志的仓库 (ext3, ext4, XFS等):
- 4. 管理员搬货前，先在**记事本 (Journal)**上写下一句话：“我现在要搬这台冰箱”。如果搬一半断电了，下次开机他只要看一眼记事本，就知道刚才冰箱没搬完，只需要处理这一个动作就行。

如何变身？

根据你的图片，我们可以通过 `mke2fs -j` 命令给原本没有日志的 ext2 加上这个“记事本”，从而把它变成 ext3。

🔍 选项解析

- **ext2：正确答案。**它是 ext 家族中老一代的文件系统，**不具备**日志功能。
- **ext3：**在 ext2 基础上增加了 **Journaling**（日志）功能。
- **ext4：**ext3 的进化版，目前 Linux 最主流的日志文件系统。
- **XFS：**高性能的 64 位日志文件系统。
- **JFS：**IBM 开发的日志文件系统（名字里的 **J** 就代表 **Journaling**）。

📊 关联表格背诵：文件系统进化论

为了帮你记牢，我们把它们排个队：

文件系统	是否有日志 (ジャーナリング)	核心创建选项 (mke2fs)
ext2	❌ なし	mke2fs (默认)
ext3	✅ あり	mke2fs -j
ext4	✅ あり	mke2fs -t ext4
XFS / JFS	✅ あり	mkfs -t [type]

✅ 正确答案

 考试小秘籍

考试中如果问到“由于停电导致文件系统损坏，哪种系统恢复最慢”，答案通常也是 **ext2**，因为它没有日志，必须进行全盘扫描。

这一知识点主要考察 Linux 中处理**タブ（Tab 键）与スペース（空格）**转换的专用工具。

expand [オプション] [ファイル名]

オプション	説明
-i	行頭のタブのみ変換
-t スペース数	1つのタブで置き換えるスペースの数を指定 (デフォルトは8)

 知识点总结

在 Linux 中，使用 **expand** 命令可以将文件中的 **タブをスペースに変換**（将 Tab 转换为个空格）。其反向操作命令（将空格转回 Tab）是 **unexpand**。

 小学生级说明：神奇的“伸缩尺”

想象你有一行文字，中间用 Tab 键留了一些大空格。

- 1. 拉长器 (**expand**):
- 2. 它像一把伸缩尺。当你按下 **expand** 按钮，它会把那个“虚”的 Tab 键占位符，实实在在地填满成好几个真正的“空格字符”。
- 3. 缩回器 (**unexpand**):
- 4. 这是它的死对头。它会把一串连续的空格重新变回一个 Tab 键。
- 5. 其他干扰项：
 - **tabchange** / **changetab** / **exchange**：这些都是出题老师编出来的“山寨”命令，Linux 系统里根本没有它们。

 选项解析

- **expand**：正确答案。专门用于将 Tab 转换为指定数量的空格（默认 8 个）。
- **unexpand**：错误。它的功能正好相反，是将**空格转换为 Tab**。
- **tabchange** / **exchange** / **changetab**：错误。不存在这些命令。

关联表格背诵：文本格式微调工具箱

为了帮你记全这些容易混淆的命令，我们把它们放在一起对比：

命令	核心功能 (日本語)	记忆口诀 (中文)
expand	タブ → スペース	Expand （扩张），把 Tab 变宽成空格。
unexpand	スペース → タブ	Un- （反向），把空格缩回 Tab。
tr	文字の置き換え・削除	Translate （转换）单个字母。
fmat	テキストを整形する	Fmat （格式化）段落宽度。

正确答案

expand

考试小秘籍

- 看到 “タブをスペースに” → 选 `expand`。
 - 看到 “スペースをタブに” → 选 `unexpand`。
 - 两个单词长得很像，记住 **Expand** 是“展开/扩张”，Tab 变成一串空格就像是被展开了一样。
- 这一知识点主要考察 **正規表現（正则表达式）** 在匹配特定字符时的应用，特别是如何识别包含「#」的行。

知识点总结

要匹配包含「#」的行，可以使用直接匹配该字符的模式，或者使用表示“以 # 开头”或“包含 # 且后跟任意内容”的正则组合。

小学生级说明：寻找带“#”号的派对邀请函

想象你在一堆信件（文件行）里找东西，你只需要那些印了“#”符号的信。

- 直接找 (`#`)：
- 这就是最简单的规则。只要行里出现了这个符号，不管是开头、中间还是结尾，都能被抓出来。
- 带箭头的找 (`^#`)：
- `^` (尖帽子) 代表 **行の先頭**。所以这个规则是找“以 # 开头”的行。在你的例子中，前三行都是。

5. 带尾巴的找 (#.*):
6. 这个规则稍微复杂点:
- # : 先找到这个号。
 - . (隐身衣): 代表任意 1 个字符。
 - * (复读机): 代表前面的东西出现 0 次以上。
 - 合起来的意思就是: “有一个 #, 后面跟着什么都行 (或者什么都不跟)”。

谁是捣乱的?

- ^[^#] : 这代表 “不以 # 开头” 的行 ([^] 是排除的意思)。
- [^#] : 这代表匹配任何 “不是 #” 的单个字符。

 选项解析

- ^# : 正确答案。匹配以 # 开头的行。
- #.* : 正确答案。匹配包含 # 且后面跟着任意 0 个或多个字符的行。
- # : 正确答案。最基础的匹配, 只要行内有 # 即可。
- ^[^#] : 错误。匹配不以 # 开头的行。
- [^#] : 错误。匹配非 # 的字符。

 正确答案 (全て選択)

- ^#
- #.*
- #

 关联表格背诵: 正则符号的 “位置感”

为了帮你记牢这几张正则表, 请重点背诵这两个位置符号:

符号	含义 (日本語)	记忆口诀
^	行の先頭	像个尖帽子戴在头 (开头) 上。
\$	行の末尾	像个小尾巴摇在最后面。
[]	いずれかの1文字	框起来选一个。
[^]	含まない1文字	框里加个尖角, 代表 “排除”。

 考试小秘籍

如果题目问的是“コメント行（注释行）”匹配，通常答案一定是 `^#`，因为 Linux 配置文件里的注释都是以 # 开头的。

这一知识点主要考察 **viエディタ（vi编辑器）** 在 **コマンドモード（命令模式）** 下的 **画面スクロール（屏幕滚动）** 指令。

 知识点总结

在 vi 的命令模式下，使用 `Ctrl + f` 可以将画面向**下方（前方）**滚动一整屏。

 选项解析与图片关联记忆

根据 vi 的操作逻辑，我们可以将选项分为“移动”和“滚动”两类：

- `Ctrl + f`：正确答案。
 - 含义：Forward（前进/下方）。将画面向下移动一屏。
- `Ctrl + b`：错误。
 - 含义：Backward（后退/上方）。将画面向上移动一屏。
- `G`：错误。
 - 含义：移动到文件的 最後行（最后一行）。
- `gg`：错误。
 - 含义：移动到文件的 先頭行（第一行）。
- `L`：错误。
 - 含义：Low。将光标移动到当前显示画面的 最下行，但不翻页。

コマンド	説明
h	1文字左へ移動
l	1文字右へ移動
k	1文字上へ移動
j	1文字下へ移動
0	行頭へ移動
^	行の先頭の文字へ移動
\$	行末へ移動
H	画面の最上行へ移動
L	画面の最下行へ移動
gg	ファイルの先頭へ移動
G	ファイルの最終行へ移動
nG	ファイルのn行目へ移動
:n	ファイルのn行目へ移動
Ctrl+b	1画面分、上方を表示
Ctrl+f	1画面分、下方を表示

 vi 画面移动指令背诵表

为了帮你像小学生一样记住这些按键，我们利用英文首字母来联想：

指令	功能 (日本語)	记忆口诀 (中文)
Ctrl + f	1画面分、下にスクロール	Forward = 向前（下）冲！
Ctrl + b	1画面分、上にスクロール	Backward = 向后（上）退。
Ctrl + d	半分、下にスクロール	Down = 下去一半。
Ctrl + u	半分、上にスクロール	Up = 上去一半。
G	最後行へ移動	Grave = 到底了（坟墓）。
gg	先頭行へ移動	go go = 出发（第一行）。

🧠 记忆大作战：翻页键的“魔力”

想象你正在看一卷长长的卷轴（文件）：

1. **f (Forward)**：你用力把卷轴向下扯了一大把，看到了后面的新内容。
2. **b (Backward)**：你觉得刚才看漏了，又把卷轴向上卷了一大把。
3. **d (Down)** 和 **u (Up)**：力气小一点，只扯动一半的内容。

✅ 正确答案

Ctrl+f

📝 考试小秘籍

考试时如果分不清 **f** 和 **b**：

- **F** 代表 **F**ront/**F**orward（前方/前进），在阅读时，前进就是往页码大的方向走，也就是**下方**。
- **B** 代表 **B**ack（后退），也就是回到**上方**。

这一知识点主要考察 `split` コマンド 在默认情况下的 **分割单位（分割单位）**。

💡 知识点总结

当使用 `split` 命令而没有通过 `-l` 选项指定行数时，系统会默认以 **1000行** 为单位对文件进行切分。

🐣 小学生级说明：自动切火腿机

想象你有一根特别长的火腿（大文件），你把它塞进了一台叫 `split` 的自动切片机里。

- 1. 如果你不调旋钮（不加选项）：
- 2. 机器内部设定了一个默认值。它每数到 **1000行** 就会“咔嚓”切一刀，把它变成一个小包装。
- 3. 切完后的名字：
- 4. 它不仅帮你切，还帮你贴标签。默认会给切出来的文件起名为 `xaa`，`xab`，`xac` ... 这样排下去。
- 5. 其他干扰项：
 - “弹出提示”：Linux 命令通常很酷，不会弹出对话框问你，你不说它就按默认办。
 - “500行”：这是老师编出来骗你的数字，记住是 **1000** 这个整大数。

选项解析

- 1000行ごとに分割される：正确答案。这是 Linux 系统的标准默认设置。
- 指定行数をたずねるプロンプトが出る：错误。它是非交互式的。
- 出力ファイルが指定されないと標準出力に表示される：错误。`split` 的目的就是生成文件，不会只在屏幕上晃一下。
- 指定しないと分割されない：错误。它会直接按默认 1000 行开切。
- 500行ごとに表示される：错误。数字不对。

关联表格背诵：常用的“数字”默认值

在 LPI 考试中，记住这些默认数字非常有用：

命令	默认操作对象	默认数量
<code>split</code>	行数	1000行
<code>head</code>	行数	10 行
<code>tail</code>	行数	10 行
<code>od -t o</code>	字节单位	4 字节

正确答案

1000行ごとに分割される

考试小秘籍

如果你想自定义行数，要记住选项是 `-l` (Lines)。例如 `split -l 500 file.txt` 才会按 500 行切。

メタキャラクタ	説明
*	0文字以上の文字列
?	任意の1文字
[]	[]内のいずれか1文字 [abc] a,b,cのどれか1文字 [a-z] 小文字のアルファベット1文字 [!abc] a,b,c以外の1文字
\$	変数
'	「'」で囲まれた部分を文字列と見なす (メタキャラクタの意味を打ち消す)
”	「”」で囲まれた部分を文字列と見なす (メタキャラクタの意味を打ち消す。 ただし「\$」「`」「¥」「”」は除く)
`	「`」で囲まれた文字列(変数の場合は 格納されている値)をコマンドと見なし、 その実行結果を文字列として返す
¥	次の文字(メタキャラクタ)の意味を 打ち消して通常の文字として扱う (エスケープする)
ディレクトリの特殊記号	
~	ホームディレクトリ
.	カレントディレクトリ
..	カレントディレクトリの1つ上の階層

这一知识点主要考察 **シェルにおける変数（Shell 中的变量）** 的引用方式。在 Linux 环境中，系统或用户定义的变量需要通过一个特殊的 **メタキャラクタ（元字符）** 来告诉 Shell：“嘿，这后面不是普通的单词，是一个变量名！”

💡 知识点总结

在 Linux Shell 中，`$` 是用于表示 **変数（变量）** 的元字符。当你想要读取变量的值（引用变量）时，必须在变量名前加上这个符号。

🔍 选项解析

- `$`：正确答案。
 - **用法**：例如你定义了变量 `NAME="Gemini"`，要打印它时必须写 `echo $NAME`。

- `?`：错误。
 - 含义：在通配符（Wildcard）中代表 **任意の1文字（任意 1 个字符）**。
- `"` (双引号)：错误。
 - 含义：用于括起字符串。它允许其中的变量（如 `$NAME`）被解析（展开）。
- `'` (单引号)：错误。
 - 含义：强引用。它会把里面所有的字符都当成普通的文字，变量 `$NAME` 在这里不会被解析。
- `\` (反斜杠)：错误。
 - 含义：**エスケープ文字（转义字符）**。用于取消紧随其后的元字符的特殊魔力。

关联表格背诵：Shell 里的“符号魔法师”

为了帮你像小学生一样记住这些符号在变量和引用中的角色，请看这张对比表：

符号	角色 (日本語)	记忆口诀 (中文)
\$	変数の参照	\$ 代表钱（变量的值很值钱）。
"	弱引用	双引号很宽容，允许变量 \$ 变身。
'	強引用	单引号很死板，\$ 在里面就是个普通符号。
\	エスケープ	橡皮擦，擦掉符号的魔力。

记忆大作战：变量的“入场券”

想象变量是一个穿着便衣的超级英雄（比如 `PATH`）：

1. 他直接走在街上（直接写 `PATH`），大家只觉得他是个普通的单词。
2. 只有当他拿出一张 `$` 形状的入场券（写成 `$PATH`），他才会瞬间变身，展示出他真正的力量（显示出一长串路径）。

正确答案

\$

考试小秘籍

考试中如果问到“如何查看上一个命令的返回状态（退出码）”，记得找 `$?`。这里的 `$` 依然是变量符号，而 `?` 则是这个特殊变量的名字。

这一知识点主要考察 **シェル変数 (Shell 变量)** 的删除操作。

💡 知识点总结

在 Linux 中，删除变量使用 `unset` 命令，且后面只需要跟 **变数名 (变量名)**，不需要加 `$` 符号。

🔍 选项解析与详细说明

- `unset LPIC`：正确答案。
 - **原因**：`unset` 是 Bash 内置的用于销毁变量或函数的命令。在执行删除操作时，我们操作的是变量的“名字”本身，所以直接写变量名 `LPIC` 即可。
- `unset $LPIC`：错误。
 - **原因**：这里的 `$` 是 **メタキャラクタ (元字符)**，作用是引用变量的值。如果 `LPIC` 的值是 `101`，那么这条命令会被解析成 `unset 101`，系统会去尝试删除一个名为 `101` 的变量，而原本的 `LPIC` 却被留了下来。
- `delvariable` / `deletevariable`：错误。
 - **原因**：这些是考试中常见的 **架空のコマンド (虚构命令)**。Linux 中并没有这些命令。

📊 变量操作“三步走”对比表

为了帮你像小学生一样记牢，我们看看变量在不同阶段的形态：

动作 (日本語)	命令示例	是否带 \$	核心逻辑 (中文)
代入 (赋值)	<code>LPIC=101</code>	なし (无)	在名为 LPIC 的盒子里装入东西。
参照 (引用)	<code>echo \$LPIC</code>	あり (有)	使用 \$ 钩子把盒子里面的值钩出来。
削除 (删除)	<code>unset LPIC</code>	なし (无)	直接把名为 LPIC 的盒子整个扔掉。

🧠 记忆大作战：名牌与垃圾桶

想象你正在管理一个储物间 (Shell 环境)：

1. **贴名牌**：你拿了一个标签写上 `LPIC` 贴在柜子上，这就是创建变量。
2. **取东西 (\$)**：你想看柜子里有什么，得用“取值钩子” `$`。
3. **撕掉名牌 (unset)**：你不需要这个柜子了，你要撕掉的是 **“LPIC” 这个名牌**，而不是撕掉“柜子里面的东西”。所以 `unset` 后面直接跟名牌的名字。

✅ 正确答案

unset LPIC

📝 考试小秘籍

题目中只要看到「削除」，就锁定 `unset`。然后检查后面有没有多余的 `$`，有的话就是诱导陷阱。

这一知识点主要考察 **シェル (Shell)** 的动作行为设定。除了管理变量，某些命令还专门用于开启或关闭 Shell 的功能开关（例如：脚本出错时立即停止、显示执行过程等）。

💡 知识点总结

在 Bash 等 Shell 中，`set` 命令不仅可以显示变量，最重要的功能是用于 **シェルのオプションを設定する**（设定 Shell 选项）。

🔍 选项解析与详细说明

- `set`：正确答案。
 - **功能**：用于设置 Shell 的执行环境选项。例如，`set -x` 可以显示命令执行的详细过程，`set -e` 可以让脚本在出错时立刻退出。此外，不带参数的 `set` 会显示当前所有的变量。
 - `export`：错误。
 - **功能**：用于将 **シェル変数（局部变量）** 提升为 **環境変数（环境变量）**，以便子进程也能访问到该变量。
 - `env`：错误。
 - **功能**：用于显示当前的 **環境変数**，或者在指定的临时环境下执行命令。
 - `bash`：错误。
 - **功能**：这是 Shell 程序本身，虽然可以通过参数启动，但它不是用来在当前会话中“设定选项”的专用命令。
-

📊 变量与选项命令“四大金刚”对比表

为了帮你像小学生一样记牢，我们把这几个长得像的命令放在一起：

命令	核心作用 (日本語)	记忆口诀 (中文)
set	オプションの設定 / 全変数の表示	Set 是“设置”，给 Shell 设置规则。
export	環境変数 への格上げ	Export 是“出口”，让变量出家门给别人看。
env	環境変数 の表示と実行	Env 是 Environment（环境）的缩写。
unset	変数や関数 の削除	Un-set 是“撤销设置”，把变量扔掉。

🧠 记忆大作战：Shell 的“性格说明书”

想象 Shell 是一个正在帮你干活的小机器人：

1. `set`：就像是打开机器人的“**设置面板**”。你可以告诉它：“干活时要仔细点（显示过程）”或者“遇到错就别干了（立即停止）”。
2. `export`：就像是给小机器人发了一张“**通行证**”，让它带给它的徒弟（子进程）看。
3. `env`：就像是查看小机器人身上贴着的“**环境标签**”（比如现在的语言、时区）。

✅ 正确答案

set

📝 考试小秘籍

如果题目问“如何开启脚本的调试模式（显示每一步运行结果）”，要想到 `set -x`；如果问“如何显示所有变量（包括局部和环境）”，也要选 `set`。

オプション	説明
allexport	新規作成・変更した変数を自動的に環境変数とする(エクスポートする)
emacs	emacsエディタと同じキーバインドにする。 (矢印キーやマウスで操作でき、Ctrl+Dで一文字削除できる、など)
ignoreeof	Ctrl+D を押してもログアウトしない設定にする
noclobber	リダイレクト演算子「>」「>&」「<>」で 既存のファイルを上書き不可に設定する
noglob	パス名展開を無効に設定する (ワイルドカードによる展開(*や?)が無効になる)
vi	viエディタと同じキーバインドにする。 (ESCキーでコマンドモードに移行し、入力時はiやAなどで編集モードへの移行が必要、など)

这一知识点主要考察的是 `set` コマンド（设置命令）如何通过特定的 オプション（选项）来改变 シェル（终端外壳）的行为。

💡 知识点总结

`set` 命令使用 `-o` 来 有効にする（开启）某个功能，使用 `+o` 来 解除（关闭）该功能。

🔍 详细解释：Shell 的“性格开关”

这就像是给你的机器人（Bash Shell）设置不同的工作模式。通常我们习惯用减号 `-` 表示开启，但在 `set` 命令里，加号 `+` 反而是用来“关掉”功能的。

2.6 核心语法

- `set -o [オプション名]`：开启某个功能。
- `set +o [オプション名]`：关闭某个功能。

2.2 表格中的常见选项背诵

根据你上传的图片，这些是考试中最常出现的“性格开关”：

オプション (选项名)	功能说明 (中文解释)	记忆口诀
<code>allexport</code>	自动把新创建或修改的变量变成 環境変数。	All + Export （全部导出）。
<code>ignoreeof</code>	防止不小心按到 Ctrl+D 就直接注销登录了。	Ignore （忽略） EOF （退出信号）。
<code>noclobber</code>	防止使用 <code>></code> 重定向时意外覆盖掉已经存在的文件。	No + Clobber （不要破坏原文件）。
<code>noglob</code>	让通配符（如 <code>*</code> 或 <code>?</code> ）失效，不再展开路径名。	No + Glob （不要展开通配符）。
<code>vi</code>	让命令行输入变得像 vi エディタ 一样可以按 Esc 切换模式。	变身 vi 编辑器模式。

📖 `man` 命令补充知识

如果你在设置这些选项时遇到了困难，想查查手册：

- `man -k`：可以按关键词 一部一致（部分匹配）搜索相关的帮助。
- セクション 1: 查看 `set` 这种 ユーザーコマンド（用户命令）的用法。

🧠 小学生级记忆法：加减号的“反直觉”

想象 Shell 是一个爱干活的小人：

- 1. `set -o` (减号)：像是在小人的待办清单上 **划掉** 了一项任务，表示“这件事我接手了，我要开始 **执行**这个规则”。
- 2. `set +o` (加号)：像是给小人 **增加** 了一层限制，让他“停下别做了”，表示**解除**这个规则。

✅ 举个栗子 (例)

如果你不想让 `Ctrl+D` 把你的终端关掉：

- 开启保护： `set -o ignoreeof`
- 取消保护： `set +o ignoreeof`

コマンド	説明
set	全てのシェル変数と環境変数を表示
declare	全てのシェル変数と環境変数を表示
env	全ての環境変数を表示
printenv	指定の、または全ての環境変数を表示

这一部分知识主要考察如何**表示（表示）**和**设置（設定）** Linux 系统中的 **变数（变量）** 以及 **シェルの動作（Shell 行为）**。

💡 知识点总结

Linux 提供了多种命令来管理变量，`set` 和 `declare` 可以查看所有变量（局部+环境），而 `env` 和 `printenv` 专门用于 **環境変数（环境变量）**。

🔍 核心图表详细拆解

我们将你上传的最后两张表格合并成两个“功能包”来记忆：

2.3 变量显示 “四剑客”

当你想要看系统里存了哪些变量时，用这四个命令：

コマンド (命令)	作用 (日本語説明)	记忆口诀 (中文)
set	全てのシェル変数と環境変数を表示	全能王：管得最宽，啥变量都报。
declare	全てのシェル変数と環境変数を表示	宣告者：和 set 一样看全部。
env	全ての環境変数を表示	外向型：只看给别人看的“环境”变量。
printenv	指定の、または全ての環境変数を表示	打印机：功能同 env，但能只看某一个。

2.2 Shell 行为设置开关

这部分属于 `set -o` 的高级玩法，用来调整 Shell 的“性格”：

- `allexport`：只要你改了变量，它就自动变成 **环境变量**（省去了手动 `export` 的麻烦）。
- `noclobber`：开启 **防覆盖** 保护。如果你用 `>` 重定向时，目标文件已经存在，它会报错拒绝执行，保护你的数据。
- `noglob`：禁止**通配符** 展开。让 `*` 和 `?` 失去变身魔力，变成普通字符。
- `ignoreeof`：忽略**退出信号**。防止你手滑按到 `Ctrl+D` 把当前的 Shell 窗口关掉。
- `vi` / `emacs`：编辑器**模式**。让命令行输入的操作感变得像对应的编辑器一样。

小学生级记忆法：变量的“大小圈子”

想象变量是学校里的学生：

1. `set` / `declare`：是**全校花名册**。不管是在教室里偷偷玩的（局部变量），还是在操场上大家都能看到的（环境变量），上面全都有。
2. `env` / `printenv`：是**运动会报名表**。上面只记录了那些要在全校面前露脸的学生（环境变量）。

✓ 考点提醒

- 想看“全部”？选 `set` 或 `declare`。
- 想看“环境”？选 `env` 或 `printenv`。
- 不想文件被覆盖？开启 `set -o noclobber`。

コマンド	説明
yy Y	カーソル位置の行をバッファにコピー
yw	カーソル位置の単語をバッファにコピー
dd	カーソル位置の行を削除し、バッファにコピー
dw	カーソル位置の単語を削除し、バッファにコピー
x	カーソル位置の文字を削除し、バッファにコピー
X	カーソル位置の左の文字を削除し、バッファにコピー
p	バッファの内容を挿入(文字や単語はカーソルの右、行はカーソルの下に挿入)
P	バッファの内容を挿入(文字や単語はカーソルの左、行はカーソルの上に挿入)
u	元に戻す(直前の編集をとりやめる)
.	繰り返す(直前の編集を再実行する)

ファイルの編集に関する主なviコマンド

这一知识点主要考察 **viエディタ**（vi编辑器）在 **コマンドモード**（命令模式）下的 **削除**（删除）操作及其单位。

💡 知识点总结

在 vi 的命令模式下，`dw` 命令用于删除从光标位置开始到 **単語の末尾**（单词末尾）的内容。

🔍 选项解析与详细说明

- `dw`：正确答案。
 - 含义：**delete word**。它会删掉当前光标所在的那个单词（或者从光标处到词尾的部分）。
- `x`：错误。
 - 含义：删除光标位置的 **1文字**（1个字符）。类似于键盘上的 `Delete` 键。
- `X`（大写）：错误。
 - 含义：删除光标 **直前（前面）** 的 1 个字符。类似于键盘上的 `Backspace` 键。
- `dd`：错误。
 - 含义：删除（剪切）光标所在的 **1行**（整行）。
- `p`：错误。
 - 含义：**paste**（粘贴）。将刚才删除（剪切）的内容粘贴到光标之后。

📊 vi 常用编辑指令背诵表

为了帮你像小学生一样记住这些按键，我们把它们按“威力大小”排个队：

指令	删除单位 (日本語)	记忆口诀 (中文)
x	1文字	小 x 是小叉号，只叉掉一个字。
dw	単語	Delete Word，删掉一个词。
dd	1行	双 d 威力大，整行都变炸。
D	行末まで	大 D 扫尾，后面全没。

🧠 记忆大作战：字母背后的含义

- `d` = delete（删除）
- `w` = word（单词）

- 所以连起来 `dw` 就是“删掉单词”。

✅ 正确答案

`dw`

📝 考试小秘籍

考试时如果看到命令前面带数字，比如 `3dw`，意思就是“一次性删掉 3 个单词”。vi 里的数字通常都代表“重复次数”。

这一知识点主要考察 **viエディタ（vi编辑器）** 在 **コマンドモード（命令模式）** 下最基础的 **1文字削除（删除单个字符）** 指令。

💡 知识点总结

在 vi 的命令模式下，`x` 指令用于删除当前光标所在的 **1文字（1个字符）**。

🔍 选项解析与详细说明

根据你提供的 vi 操作表，我们可以对比这几个相似指令的区别：

- `x`：正确答案。
 - **功能**：删除光标所在位置的那个字符。相当于普通编辑器的 `Delete` 键。
- `X`（大写）：错误。
 - **功能**：删除光标 **直前（前面）** 的一个字符。相当于普通编辑器的 `Backspace` 键。
- `dw`：错误。
 - **功能**：**delete word**。删除从光标开始的一个 **単語（单词）**。
- `d`：错误。
 - **功能**：这通常是一个“组合键”的前缀。单独按 `d` 不会有动作，它在等待你告诉它删什么（比如 `dd` 删行，`dw` 删词）。
- `p`：错误。
 - **功能**：**paste**。将删除（剪切）的内容 **貼り付け（粘贴）** 到光标后面。

📊 vi 基础编辑指令背诵表

为了帮你像小学生一样记牢，请看下表：

指令	动作 (日本語)	记忆口诀 (中文)
x	1文字削除	小 x 像个小叉号，叉掉当前的字。
X	直前の1文字削除	大 X 劲儿大，反手叉掉前面的字。
dd	1行削除	连按两次 d，整行都消失。
u	直前の操作を取り消す	undo，后悔药，撤销刚才的操作。

小学生级记忆法：贪吃蛇

想象光标是一个“贪吃蛇”：

- 1. 按下 **x**：它张开小嘴，把 **肚子下面** 压住的那个字母吃掉了。
- 2. 按下 **X**：它向后甩尾巴，把 **屁股后面** 的那个字母扫掉了。

✅ 正确答案

x

📝 考试小秘籍

vi 的指令通常是区分大小写的。在 LPI 考试中，小写通常是“向后/向下/当前”操作，大写通常是“向前/向上/反向”操作。

- 小 **x** 删当前。
- 大 **X** 删前面。

这一知识点主要考察 **viエディタ (vi编辑器)** 在 **コマンドモード (命令模式)** 下的 **コピー (复制)** 操作，即如何将内容放入 **バッファ (缓冲区)**。

💡 知识点总结

在 vi 中，**yy** 用于复制 (Yank) 当前行。如果要复制多行，可以在命令前加上数字，因此 **5yy** 表示将包括当前行在内的 **5行内容复制到缓冲区**。

选项解析与详细说明

根据你提供的 vi 操作指南，我们可以分析各选项的功能：

- **5yy**：正确答案。
 - 含义：**yank yank**（复制行）。前面的数字 **5** 表示重复 5 次。执行后，这 5 行会被存入缓冲区，等待被粘贴。
- **5dd**：错误。
 - 含义：**delete delete**（删除行）。虽然它也会将内容放入缓冲区（类似于“剪切”），但它的主要动作是 **削除（删除）**，不符合题目仅要求“复制”的意图。
- **5p**：错误。
 - 含义：**paste**（粘贴）。将缓冲区的内容在光标后 **貼り付け（粘贴）** 5 次。
- **5P**（大写）：错误。
 - 含义：在光标前粘贴 5 次。
- **5x**：错误。
 - 含义：删除当前光标位置开始的 **5个文字**。

vi 行操作指令背诵表

为了帮你像小学生一样记牢，请对比这组“行”操作：

指令	动作 (日本語)	记忆口诀 (中文)
yy	1行コピー (ヤンク)	Yank 是“拽”，把这行拽进缓冲区。
dd	1行削除 (切り取り)	Delete 是“删”，整行都变没。
p	貼り付け (後ろ)	Paste 是“贴”，贴在屁股后面。
P	貼り付け (前)	大 P 劲儿大，贴在脑门前面。

小学生级记忆法：复印机

想象 vi 编辑器里有一台复印机：

1. **yy**：就像你对着一行字喊“**予（Yǔ）备——印！**”。它会扫描这行字并存进内存。
2. **5yy**：就是连喊 5 声，把接下来的 5 行都扫描进去。
3. **注意**：这时候屏幕上什么都不会发生，直到你按下 **p** 把它们印出来。

✅ 正确答案

5yy

📝 考试小秘籍

在 vi 的世界里，“复制”不叫 Copy，而叫 **Yank**（提取）。所以只要看到题目要求“コピー”，选项里一定要找带 **y** 的指令。

这一知识点主要考察 **viエディタ（vi编辑器）** 在 **コマンドモード（命令模式）** 下执行 **貼り付け（粘贴）** 操作时的位置差异。

💡 知识点总结

在 vi 中，使用 **p** 或 **P** 将 **バッファ（缓冲区）** 的内容贴出。其中，**P**（大写）用于将内容粘贴在光标的 **左侧（或上方）**。

🔍 选项解析与详细说明

根据你提供的操作对照表，我们可以发现粘贴指令分为大小写两种：

- **P（大写）：正确答案。**
 - **功能**：在光标的 **左侧**（针对字符/单词）或 **上方**（针对整行）进行粘贴。
- **p（小写）：错误。**
 - **功能**：在光标的 **右侧**（针对字符/单词）或 **下方**（针对整行）进行粘贴。
- **yy：错误。**
 - **功能**：**コピー（复制）** 当前行。
- **dd：错误。**
 - **功能**：**削除（删除）** 当前行。
- **dw：错误。**
 - **功能**：删除当前单词。

vi 粘贴位置对比表

为了帮你像小学生一样记牢，请对比这两者的“势力范围”：

指令	粘贴方向 (日本語)	记忆口诀 (中文)
p (小写)	後ろ / 下に貼り付け	post (在后)，贴在后面。
P (大写)	前 / 上に貼り付け	Prior (在前)，大写的更有力，抢占前面。

小学生级记忆法：插队的小人

想象你正在排队（一行字），你的光标是当前排在中间的小人：

- 按下 **p**：新内容很礼貌，站在你的 **后面 (右边)**。
- 按下 **P**：大写的 **P** 就像一个很霸道的人，直接 **插队** 站到了你的 **前面 (左边)**。

正确答案

P

考试小秘籍

在 LPI 考试中，vi 的大小写通常代表“相反”的方向。

- 小写 **p** = 后面（向右/向下）。
- 大写 **P** = 前面（向左/向上）。
- 记住这个规律，即便遇到不熟悉的指令也能猜出大概。

这一知识点主要考察使用 **cut コマンド**（裁剪命令）按 **文字（字符位置）** 提取数据的方法。

cut [オプション] [ファイル名]

オプション	説明
-c 文字数	抽出する文字位置を指定
-d 区切り文字	区切り文字を指定(デフォルトはタブ)
-f フィールド	抽出するフィールドを指定 n → n番目のフィールド n,m → n番目とm番目のフィールド

 知识点总结

当需要从文件每一行中提取特定位置的 **文字（字符）** 时，应使用 `cut -c` 选项（`c` 代表 **Character**）。

 选项解析与详细说明

- `cut -c 2 /etc/passwd`：正确答案。
 - 功能：提取每行的第 2 个 **Character**（文字）。
- `cut -f 2 /etc/passwd`：错误。
 - 功能：按 **Field**（列/字段）提取。默认以 Tab 键分割，而 `/etc/passwd` 是用冒号 `:` 分割的，所以不加 `-d` 无法正确工作。
- `cut -d 2 /etc/passwd`：错误。
 - 功能：`-d` 是用来指定 **Delimiter**（分隔符）的，后面应该跟一个字符（如 `:`），而不是数字。
- `cut -a` / `cut -p`：错误。
 - 功能：`cut` 命令中没有这两个标准选项。

 `cut` 命令常用选项背诵表

为了帮你像小学生一样记牢，我们把这几个关键字母对齐：

选项 (日本語)	含义 (中文解释)	记忆口诀
-c	文字単位で指定	Character = Cut 文字。
-f	フィールド単位で指定	Field = Follow 字段 (列)。
-d	デリミタ (区切り文字) を指定	Delimiter = Divide 分隔符。

🧠 小学生级记忆法：剪刀手的“剪法”

想象你手里有一把叫 `cut` 的神奇剪刀：

1. `cut -c`：像是一把**细长的小剪刀**，专门对着第几个字咔嚓剪下去。
2. `cut -f`：像是一把**大剪刀**，它要看准“缝隙”（分隔符）再剪。如果你不告诉它缝隙在哪（用 `-d` 指定），它就找不到地方下剪。

✅ 正确答案

`cut -c 2 /etc/passwd`

📝 考试小秘籍

题目中如果说的是“**2番目の文字**”（第2个字符），首选 `-c`；如果说的是“**2番目のフィールド**”（第2个字段/列），则一定要找 `-f`。

这一知识点主要考察在 **viエディタ（vi编辑器）** 的 **コマンドモード（命令行模式）** 中如何临时执行 **外部コマンド（外部命令）**。

💡 知识点总结

在 vi 中，通过输入 `:!` 后跟命令名，可以实现在 **終了することなく（不退出编辑器）** 的情况下运行 Linux 系统的外部命令。

🔍 选项解析与详细说明

- `:!ls`：正确答案。
 - **功能**：`:` 进入命令行模式，`!` 告诉 vi “我要运行一个 Shell 命令”，`ls` 则是具体的外部命令。执行后，屏幕会暂时切换到终端显示结果，按回车键后回到 vi 编辑界面。
- `:ls`：错误。
 - **功能**：这是 vi 的内部命令，用于列出当前 vi **バッファ（缓冲区）** 中打开的文件列表，而不是显示磁盘上的目录文件。
- `!:ls`：错误。
 - **功能**：格式错误，`!` 必须跟在 `:` 后面。
- `/ls` 和 `?ls`：错误。
 - **功能**：这两个是 **检索（搜索）** 命令。`/` 向下搜索字符串 "ls"，`?` 向上搜索字符串 "ls"。

vi 外部交互指令背诵表

为了帮你像小学生一样记牢，记住这个“惊叹号”的魔力：

指令格式	作用 (日本語)	记忆口诀 (中文)
:! 命令	外部コマンドを実行する	!是惊叹号，代表“跳出去”执行。
:r! 命令	命令の実行結果を読み込む	read + !，把结果抓进文件里。
:sh	一時的にシェルに戻る	shell，暂时去外面透透气。

小学生级记忆法：任意门

想象 vi 编辑器是一个密封的房间：

1. **:!ls**：你在墙上画了一个惊叹号形状的“任意门”。你推开门看了一眼外面的风景（**ls** 查目录），看完后门一关，你依然稳稳地坐在 vi 的房间里，文件也没丢，也没关。

正确答案

:!ls

考试小秘籍

只要题目问“不退出 vi”且要“执行系统命令/查看目录/查看日期”，选项里一定要找带有 **:!** 组合的那个。

コマンド	説明
/文字列	指定した文字列をカーソル位置からファイルの末尾（前方・進行方向）に向かって検索
?文字列	指定した文字列をカーソル位置からファイルの先頭（後方・逆方向）に向かって検索
n	次を検索
N	次を検索（逆方向）

这一知识点主要考察 **viエディタ（vi编辑器）** 的 **検索（搜索）** 及其后续操作指令。

知识点总结

在 vi 中，使用 **n** 继续搜索 **順方向（相同方向）** 的下一个匹配项，而使用 **N** 则用于搜索 **逆方向（相反方向）** 的下一个匹配项。

 选项解析与详细说明

根据你提供的图片资料，我们可以清晰地看到搜索指令的逻辑：

- **N**：正确答案。
 - 功能：次を検索（逆方向）。题目中已经使用了 `/sbin` 向文件末尾方向搜索，因此想要“往回找”上一个匹配项，必须使用大写的 **N**。
- **n**：错误。
 - 功能：次を検索。它会沿着原本的方向（向文件末尾）继续找下一个 "sbin"。
- **/**：错误。
 - 功能：这是发起 順方向（向下）搜索的起始指令。
- **?**：错误。
 - 功能：这是发起 逆方向（向上）搜索的起始指令。
- **M**：错误。
 - 功能：这是光标移动命令，将光标移动到当前屏幕的 **Middle（中间）**，与搜索无关。

 vi 搜索指令背诵表

为了帮你像小学生一样记牢，请看下表：

指令	动作 (日本語)	记忆口诀 (中文)
/ 文字列	末尾に向かって検索	/ 像个滑梯，向下搜索。
? 文字列	先頭に向かって検索	? 是问号，回头问问上面有什么。
n	次を検索（順方向）	n ext，继续按原计划走。
N	次を検索（逆方向）	No! 我走反了，我要往回找。

 小学生级记忆法：追小狗

想象你在森林里追踪一只叫 "sbin" 的小狗：

1. **/**：你发现小狗往山下跑了，你开始向下追。
2. **n**：你继续往山下走，寻找下一处脚印。
3. **N**：你突然觉得可能追过了，于是掉头往山上找。大写的 **N** 就像一个巨大的掉头路标。

📝 考试小秘籍

这个知识点非常喜欢考“逆方向”。记住：

- 小写 **n** = 顺着来。
- 大写 **N** = 反着来。
- 无论你最开始是用 **/** 还是 **?** 发起的搜索，大写 **N** 永远代表与当前方向**相反**。

这一知识点主要考察 **viエディタ（vi编辑器）** 的 **保存（保存）** 与 **終了（退出）** 指令的不同组合。

コマンド	説明
:q	viを終了 ただし、内容が変更されている場合は、警告が出て終了できない
:q!	viを強制終了 内容が変更されている場合でも保存せず終了
:w	編集内容を保存
:wq :x ZZ	編集内容を保存しviを終了
:wq!	編集内容を保存しviを強制終了 読み取り専用のファイルでも強制的に保存して終了（強制保存に失敗したらviは終了しない）
:e ファイル名	viを終了せずに、現在開いているファイルを閉じて別ファイルを開く ただし、内容が変更されている場合は、警告が出て別ファイルを開けない
:e! [ファイル名]	viを終了せずに、現在開いているファイルを閉じて別ファイルを開く 内容が変更されている場合は、現在開いているファイルを最後に保存した状態に戻してから別ファイルを開く ファイル名を省略した場合 (:e! のみ) は、現在開いているファイルを最後に保存した状態に戻す

在 vi 中，想要实现“保存并退出”有多种方式，常见的包括末行模式下的 `:wq`、`:x` 以及命令模式下的 `ZZ`。

选项解析与详细说明

根据你提供的操作对照表，我们来逐一筛选正确答案：

- `:wq`：正确答案。
 - 含义：**w**rite（保存）+ **q**uit（退出）。这是最标准的组合命令。
- `:wq!`：正确答案。
 - 含义：强制保存并退出。即使文件是只读的，只要你是所有者，它也会尝试强制写入并退出。
- `:x`：正确答案。
 - 含义：与 `:wq` 功能几乎相同。如果文件被修改过，则保存并退出；如果没改过，则直接退出。
- `ZZ`：正确答案。
 - 含义：这是在 **コマンドモード（命令模式）** 下直接输入的快捷键（注意是大写），效果等同于 `:x`，即保存并退出。
- `:q`：错误。
 - 含义：仅退出。如果文件被修改过，vi 会报错提示你尚未保存。
- `:w`：错误。
 - 含义：仅保存。执行后你依然停留在 vi 编辑界面中。

vi 退出指令背诵表

为了帮你像小学生一样记牢，请看这个“存活”逻辑表：

指令	动作 (日本語)	记忆口诀 (中文)
<code>:wq</code>	保存して終了	Write (写) + Quit (走), 写完就走。
<code>ZZ</code>	保存して終了	ZZ 像睡觉, 保存好了去睡觉。
<code>:x</code>	保存して終了	x 是“及格线”，改完存好才给过。
<code>:q!</code>	保存せずに終了	Quit (走) + ! (强制), 哪怕没存也硬要走。

小学生级记忆法：写完作业去玩耍

想象你在学校写作业（编辑文件）：

1. `:wq`：你跟老师汇报：“我**写(w)完了，现在请求(q)**回家。”
 2. `ZZ`：你写完作业直接把本子一合，打了个大大的哈欠(ZZ)，直接回家睡觉了。
 3. `:q!`：你作业写一半不想写了，直接强行(!)逃课(q)，刚才写的一点都没记在档案里。
-

✅ 正确答案

`:x, :wq!, :wq, ZZ`

📝 考试小秘籍

题目要求“**全て選べ**”（多选）时，一定要把这三个老伙伴都找出来：`:wq`、`:x` 和 `ZZ`。如果题目中有 `!`，只要前面带 `w`，通常也是保存退出的意思。

💡 知识点总结

在 vi 中，想要退出编辑器，必须根据是否需要 **保存（保存）** 或是否需要 **強制（强制）** 执行来选择不同的指令组合。

🔍 核心指令详细拆解

根据你提供的图片资料，我们可以将退出指令分为以下几类：

2.3 正常退出（保存并走人）

- `:wq` | `:x` | `ZZ` :
 - **功能**：編集内容を保存しviを終了（保存编辑内容并退出 vi）。
 - **区别**：`:wq` 和 `:x` 在末行输入，`ZZ` 是在命令模式下直接按大写字母。
- `:wq!` :
 - **功能**：强制保存并退出。即使是只读文件，也会尝试保存后退出。

2.2 仅退出（不保存或报错）

- `:q` :
 - **功能**：viを終了（退出 vi）。
 - **限制**：如果内容有变动，会发出警告且无法退出。
- `:q!` :
 - **功能**：viを強制終了（强制退出 vi）。
 - **特点**：即使内容变了也不保存，直接丢弃修改退出。

2.3 仅保存（不退出）

- `:w` :
 - 功能：編集内容を保存（保存编辑内容）。执行后仍停留在编辑界面。

2.4 切换与恢复（不退出 vi）

- `:e 文件名` : 关闭当前文件并打开新文件。如果当前文件没存，会报警。
- `:e!` :
 - 功能：不保存当前修改，直接恢复到文件 最後に保存した状態（最后一次保存的状态）。这是“彻底反悔”的绝招。

vi 退出逻辑判定表

为了帮你像小学生一样记牢，我们可以通过这张逻辑图来选命令：

你的目标	文件改了吗？	推荐命令
我要存了再走	改了	<code>:wq</code> 或 <code>ZZ</code>
我不存，直接走	改了	<code>:q!</code>
我没改，直接走	没改	<code>:q</code>
我改坏了，重来	改了	<code>:e!</code>

小学生级记忆法：惊叹号的“霸道”

在 Linux 的世界里，`!` (惊叹号) 就像是一个霸道的“通行证”：

1. `:q` 是很有礼貌地请示：“老师我可以回家吗？” 如果作业（内容修改）没写完，老师会拒绝。
2. `:q!` 是你直接把桌子掀了，大喊一声：“我就要走！” 老师拦不住你，但你的作业也就白写了。

这一知识点主要考察 **viエディタ（vi编辑器）** 在 **終了することなく（不退出）** 的情况下，仅执行 **保存（保存）** 操作的指令。

知识点总结

在 vi 的末行模式中，使用 `:w` 命令可以实现 **編集内容を保存（保存编辑内容）** 且继续留在编辑界面中。

选项解析与详细说明

根据你提供的操作对照表，我们可以对比这些命令的执行结果：

- **:w**：正确答案。
 - 含义：write（写入/保存）。它只负责把内存中的修改同步到磁盘文件里，操作完成后，你依然可以继续编辑。
- **:wq**：错误。
 - 含义：write（保存）+ quit（退出）。虽然它也保存了，但它会 **退出编辑器**，不符合题目“不退出”的要求。
- **ZZ**：错误。
 - 含义：命令模式下的快捷键，等同于保存并退出，同样会关闭编辑器。
- **:x**：错误。
 - 含义：保存（如果内容已更改）并退出。
- **:q**：错误。
 - 含义：quit（退出）。这只是尝试退出，而且如果文件没保存，它还会报错。

vi 保存与退出指令对比表

为了帮你像小学生一样记牢，请看这组“动作”组合：

指令	动作 (日本語)	结果 (会退出吗?)	记忆口诀 (中文)
:w	保存	いいえ (不退出)	wait，保存一下等会儿再写。
:wq	保存して終了	はい (退出)	work + quit，干完活走了。
:q!	保存せずに終了	はい (退出)	quit + !，不干了，强行跑路。

小学生级记忆法：写日记

想象你正在写日记：

1. **:w**：你写完一段，怕停电丢失，赶紧点了一下 **“保存”** 按钮。你还想接着写，所以日记本还是摊开的。
2. **:wq**：你写完了，点了一下 **“保存并关闭”**。日记本合上放进了抽屉。
3. 题目要求：是“不关日记本”，所以只能点那个单独的 **:w**。

✓ 正确答案

:w

✎ 考试小秘籍

题目中的关键眼是「終了することなく」（不退出）。

- 只要看到“不退出”，答案一定只有 `:w` 相关的。
- 只要看到“保存并退出”，答案会有 `:wq`，`:x`，`ZZ`。

这一知识点主要考察 **viエディタ（vi编辑器）** 在 **コマンドモード（命令模式）** 下进行 **行移動（行移动）** 的快捷操作。

コマンド	説明
h	1文字左へ移動
l	1文字右へ移動
k	1文字上へ移動
j	1文字下へ移動
0	行頭へ移動
^	行の先頭の文字へ移動
\$	行末へ移動
H	画面の最上行へ移動
L	画面の最下行へ移動
gg	ファイルの先頭へ移動
G	ファイルの最終行へ移動
nG	ファイルのn行目へ移動
:n	ファイルのn行目へ移動
Ctrl+b	1画面分、上方を表示
Ctrl+f	1画面分、下方を表示

💡 知识点总结

在 **vi** 中，可以通过在 `G` 指令前加上行号，或者在末行模式（输入冒号后）直接输入行号来实现快速跳转。

🔍 选项解析与详细说明

- `5G`：正确答案。

- **功能：**在命令模式下，输入数字后跟大写 **G**，光标会直接跳转到对应的行号。例如 **1G** 跳转到首行，**G**（不带数字）跳转到末行。
- **:5**：正确答案。
 - **功能：**这是末行模式的跳转方式。输入 **:** 后直接跟数字 **5** 并回车，光标会移动到第 5 行。
- **5j**：错误。
 - **功能：****j** 是向下移动一行。**5j** 表示将光标从当前位置 **向下移动 5 行**，而不是移动到文件的“第 5 行”。
- **5k**：错误。
 - **功能：****k** 是向上移动一行。**5k** 表示将光标从当前位置 **向上移动 5 行**。
- **5h**：错误。
 - **功能：****h** 是向左移动一个字符。**5h** 表示将光标 **向左移动 5 个字符**。

 vi 光标移动指令背诵表

为了帮你像小学生一样记牢，请对比这两种“跳转”逻辑：

跳转目标 (日本語)	命令模式指令	末行模式指令	记忆口诀 (中文)
指定した行	数字G	:数字	G 是 Go ，去第几行。
ファイルの先頭	1G 或 gg	: 1	1 号位是起跑点。
ファイルの末尾	G	: \$	大 G 到底，直接跳尾部。

 小学生级记忆法：电梯与楼梯

想象这个文件是一栋大楼：

- 5G 和 :5**：就像是坐“**电梯**”。你按个 5，它直接把你送到 5 楼，不管你刚才在几楼。
- 5j 和 5k**：就像是走“**楼梯**”。你必须先知道你在几楼，然后向上走几层（**k**）或者向下走几层（**j**）。如果你在 10 楼，按 **5j** 你会去 15 楼，而不是 5 楼。

 正确答案

:5, 5G

考试小秘籍

题目要求“**全て選択**”（多选）时，一定要记得 vi 跳转行号有“**直接按键 (G)**”和“**冒号命令 (:)**”两种方式。

这一知识点主要考察的是 **テキスト整形（文本格式化）** 命令。在处理长段落时，某些命令可以根据指定的 **最大文字数（最大字符数）** 自动进行换行处理。

知识点总结

fmt 命令用于对文本进行格式化（Formatting），它最核心的功能就是通过合并或拆分行，使 **1行あたりの最大文字数** 保持在指定的范围内（默认为 **75** 个字符）。

选项解析与详细说明

- **fmt**：正确答案。
 - **功能**：整理文本段落的格式。例如，使用 `fmt -w 40 file.txt` 可以将文件内容调整为每行最多 **40** 个字符。
- **pr**：错误。
 - **功能**：用于 **印刷用（打印用）** 的格式转换。它会给文本添加页码、标题和页眉页脚，或者进行多列排版，但不主要用于控制每行的字数换行。
- **sort**：错误。
 - **功能**：对文本内容进行 **並べ替え（排序）**，例如按字母表或数字大小排序。
- **nl**：错误。
 - **功能**：为文本行添加 **行番号（行号）（Number Lines）**。
- **uniq**：错误。
 - **功能**：用于删除或报告文本中 **重複（重复）** 的行（通常需要先执行 `sort`）。

文本处理命令功能对比表

为了帮你像小学生一样记牢，我们看看这些命令各自的“特长”：

命令	核心作用 (日本語)	记忆口诀 (中文)
fmt	テキストを整形する	Format （格式化），把长句子折叠整齐。
pr	印刷用 の整形	Print （打印），加页眉页脚准备打印。
nl	行番号 を付加	Number Lines （行号），每一行前面加数字。
sort	行を 並べ替える	Sort （排序），从 A 到 Z 排排坐。
uniq	重複行 を削除	Unique （唯一），消灭重复的行。

小学生级记忆法：裁缝铺

想象你有一段长短不一的布料（文本）：

- fmt**：就像一个**裁缝**。你告诉他：“我每行只要这么宽（指定最大字数）”，他就会把长的剪断，把短的接上，让整块布看起来整整齐齐。
- nl**：就像一个**记分员**。他在每块布旁边写上：1号、2号、3号……

正确答案

fmt

考试小秘籍

如果题目中提到「幅（宽度）」或「最大文字数」，首先要联想到 **fmt**。与之相对，如果提到「ヘッダ（页眉）」或「ページ（页面）」，则通常是指 **pr**。